

Web Scraping

웹에서 정보를 자동으로 수집하기!

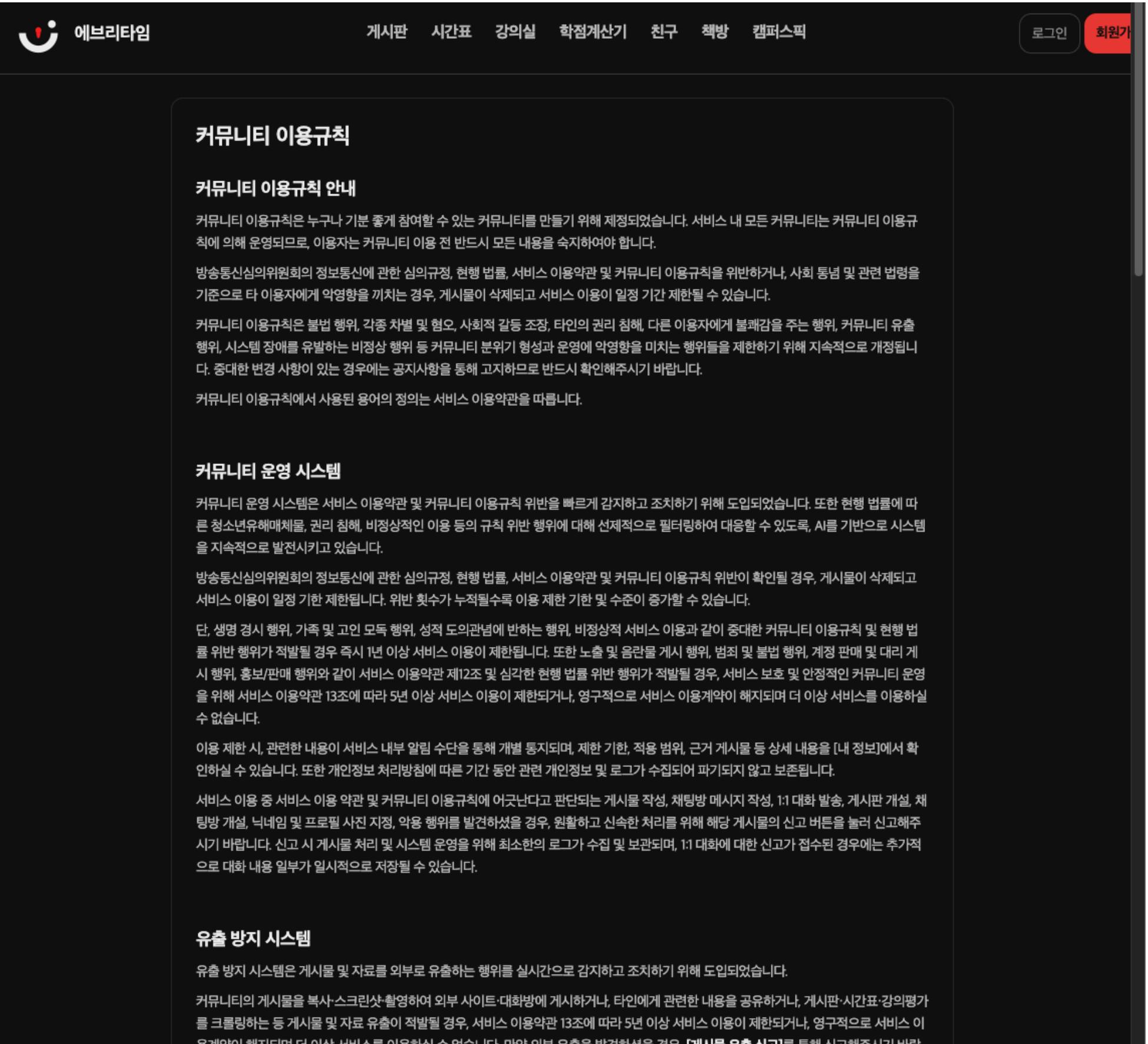
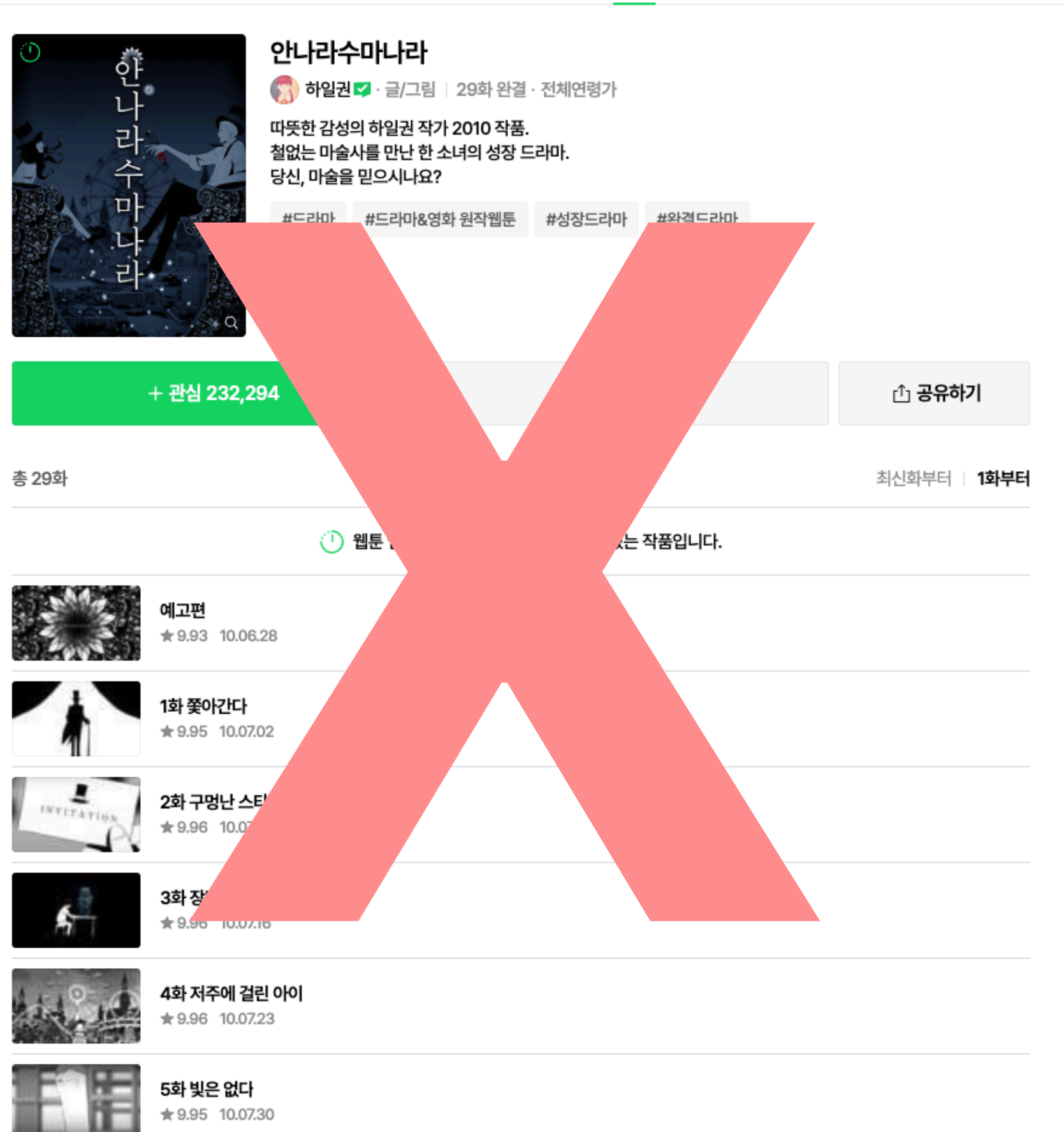
주의사항

스크래핑 전에 스크래핑이 가능한 사이트인지 확인!

사이트 base url 끝에 robots.txt 를 붙여 내용 확인

```
User-agent: *
Allow: /

Disallow: /webtoon/list?*&page=
Disallow: /webtoon/detail
Disallow: /bestChallenge/list?*&page=
Disallow: /bestChallenge/detail
Disallow: /challenge/list?*&page=
Disallow: /challenge/detail
Disallow: /business
Disallow: /community
Disallow: /creators
Disallow: /mypage
Disallow: /notice
Disallow: /curation
Disallow: /artistTitle
Disallow: /event
Disallow: /search
Disallow: /external
```



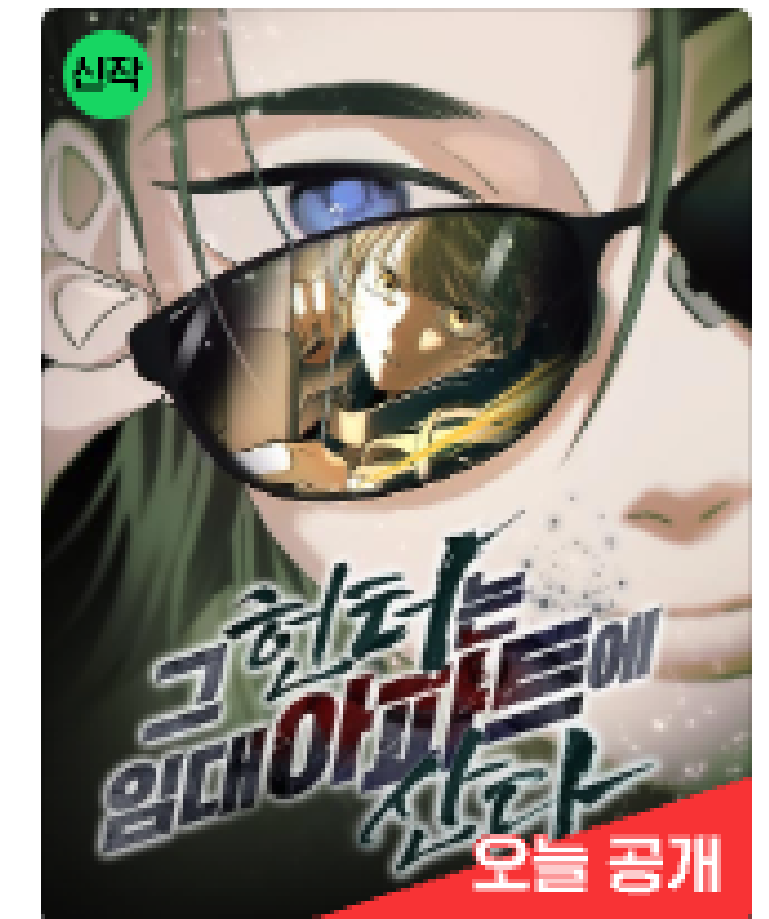
도전과제 풀어보기! 🚀

🚀 도전과제 1

앞선 “네이버 웹툰 예제”에서 제목만 가져왔습니다.

✅ 소스를 수정해서 제목 뿐만 아니라 **작가이름**을 가져와 보세요! ex) 제목 / 작가명
hint💡) 작가를 의미하는 writers 는 json 객체 입니다!

```
▼ {titleList: [{titleId: 837903, titleName: "그 헐터는 임대 아파트에 산다", author: "김형사 / 진보람",...}]
  ▼ titleList: [{titleId: 837903, titleName: "그 헐터는 임대 아파트에 산다", author: "김형사 / 진보람",...}]
    ▼ 0: {titleId: 837903, titleName: "그 헐터는 임대 아파트에 산다", author: "김형사 / 진보람",...}
      adult: false
      author: "김형사 / 진보람"
      bestChallengeLevelUp: false
      bm: false
      finish: false
      new: true
      novelOriginAuthors: [{id: 388041, name: "진보람"}]
      openToday: true
      painters: [{id: 388040, name: "김형사"}]
      potenUp: false
      rest: false
      starScore: 8.8517
      thumbnailUrl: "https://image-comic.pstatic.net/webtoon/837903/thumbnail/thumbnail.jpg"
      titleId: 837903
      titleName: "그 헐터는 임대 아파트에 산다"
      up: true
      viewCount: 0
      writers: [{id: 388040, name: "김형사"}]
        ▼ 0: {id: 388040, name: "김형사"}
          id: 388040
          name: "김형사"
      1: {titleId: 837904, titleName: "흑막 공작의 애완새", author: "몽찌 / 용사 / 유리파도",...}
```



UP 그 헐터는 임대 아파...
김형사 / 진보람
★ 8.85

도전과제 2

✅ 정렬이 “업데이트” 순으로 되어 있습니다. 인기순, 업데이트, 조회순, 별점순에 대한 데이터도 요청해 보세요!
hint💡) [Network] 탭에서 JSON 요청 URL 을 잘 살펴보세요!

URL 의 구성요소(복습)

{프로토콜}://{호스트(아이피 주소, 도메인 네임)}:{포트번호}/{... 리소스 경로 ...}?{쿼리스트링}

<https://~~~.com/~/general?order=POPULARITY&page=1> ← =파라미터를 전달

<https://comic.naver.com/webtoon?tab=new> ← 아무런 파라미터를 전달하지 않음

JSON 요청 URL : <https://comic.naver.com/api/webtoon/titlelist/new?order=update>

update : 업데이트순 / **user** : 인기순 / **view** : 조회순 / **star** : 별점순

이번주 ...

1. 이미지(파일) **다운로드** 해보기

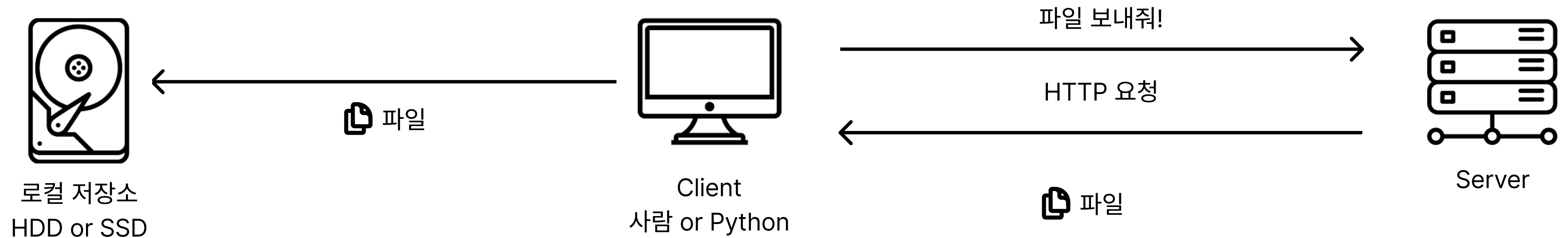
2. **로그인**이 필요한 사이트 스크래핑하기

파일을 다운로드 해보자!

파일 URL 이용해보기! 📁

‘파일 다운로드’ 무엇일까?

1. 파일을 서버에게 요청한다.
2. 서버로 파일을 받는다.
3. 받은 파일에 대한 내용을 로컬 저장소에 기록한다.



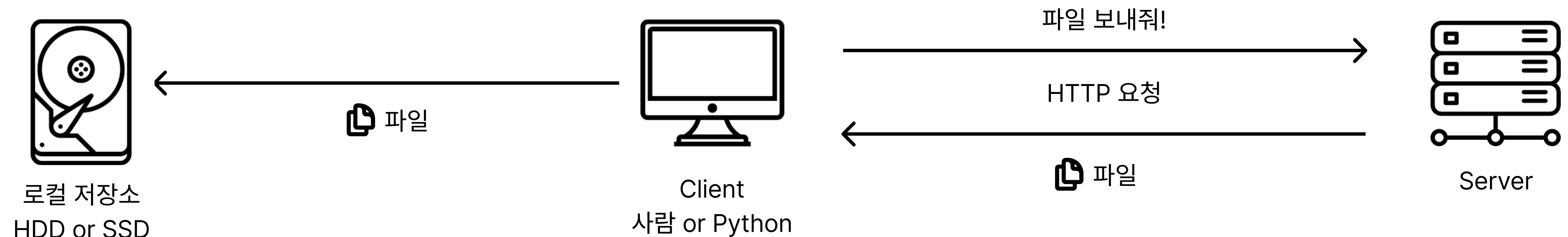
디렉터리 만들기

파일을 만들때 "어디"에 만들지 정하는 것이 중요합니다.

예를 들어, C:\test\image.jpg 라고 정하면 C 드라이브 test 폴더에 image.jpg 라는 사진을 저장하게 됩니다.
만약에, test 폴더가 없으면 프로그램은 에러가 나게 됩니다.

```
~/FLAG-Web-Scraping-Study > on main !2 ?4 python w3/w3-0.py
Traceback (most recent call last):
  File "/Users/jeong-uchang/FLAG-Web-Scraping-Study/w3/w3-0.py", line 12, in <module>
    with open("output/naver.ico", "wb") as file:
FileNotFoundError: [Errno 2] No such file or directory: 'output/naver.ico'
```

```
if not os.path.exists("output"):
    os.makedirs("output")
```



그럼 이것 Python 에서 어떻게 할까?

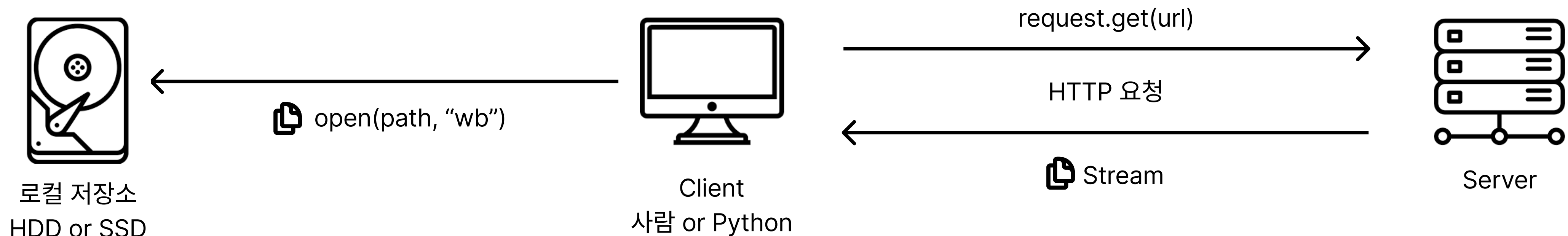
1. 파일을 서버에게 요청한다.
2. 서버로 파일을 받는다.
3. 받은 파일에 대한 내용을 로컬 저장소에 기록한다.

```
import requests
import os

# 파일 URL
url = "https://www.naver.com/favicon.ico"

# GET 요청 보내기
response = requests.get(url)

# 요청이 성공했는지 확인
if response.status_code == 200:
    # 파일로 저장
    if not os.path.exists("output"):
        os.makedirs("output")
    with open("output/naver.ico", "wb") as file:
        file.write(response.content)
    print("파일 다운로드 완료")
else:
    print(f"다운로드 실패: {response.status_code}")
```



네이버 신작 웹툰 커버 이미지 다운로드 받기

파일 URL 을 가져오자

```
data = json.loads(json_str)
```

```
file_url = data[...][...]["thumbnailUrl"]
```

```
▼ {titleList: [{titleId: 837903, titleName: "그 헛터는 임대 아파트에 산다", author: "김형사 / 진보람",...}]
  ▼ titleList: [{titleId: 837903, titleName: "그 헛터는 임대 아파트에 산다", author: "김형사 / 진보람",...}]
    ▼ 0: {titleId: 837903, titleName: "그 헛터는 임대 아파트에 산다", author: "김형사 / 진보람",...}
      adult: false
      author: "김형사 / 진보람"
      bestChallengeLevelUp: false
      bm: false
      finish: false
      new: true
      ▶ novelOriginAuthors: [{id: 388041, name: "진보람"}]
      openToday: true
      ▶ painters: [{id: 388040, name: "김형사"}]
      potenUp: false
      rest: false
      starScore: 8.8517
      thumbnailUrl: "https://image-comic.pstatic.net/webtoon/837903/thumbnail/thumbnai
      titleId: 837903
      titleName: "그 헛터는 임대 아파트에 산다"
      up: true
      viewCount: 0
      ▼ writers: [{id: 388040, name: "김형사"}]
        ▼ 0: {id: 388040, name: "김형사"}
          id: 388040
          name: "김형사"
      ▶ 1: {titleId: 837904, titleName: "흑막 공작의 애완새", author: "몽찌 / 용사 / 유리파도",...}
```

네이버 신작 웹툰 커버 이미지 다운로드 받기

파일 URL 을 가져오자

```
file_url = data['titleList'][0]["thumbnailUrl"]
```

```
response = rq.get(file_url)
```

```
with open(path_to_save, 'wb') as file:
```

```
    file.write(response.content)
```

⇒ **403** - Forbidden Error : 서버에서 특정한 이유로 클라이언트의 요청을 거부함

⇒ **User-Agent**(웹페이지의 접근하는 브라우저 정보) 설정

```
headers = {'User-Agent': '복사한 값'}
```

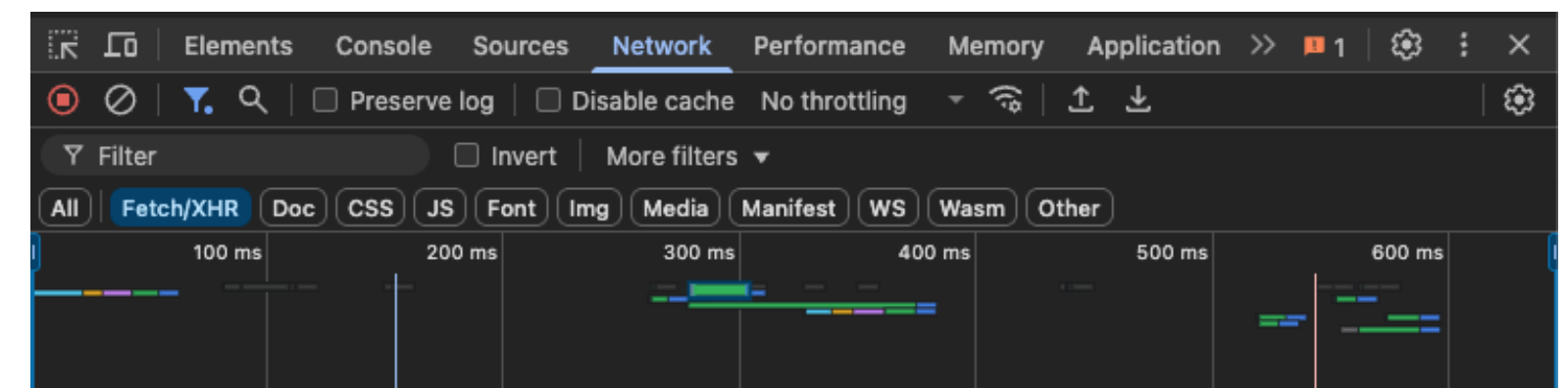
```
response = rq.get(file_url, headers = headers)
```

```
with open(path_to_save, 'wb') as file:
```

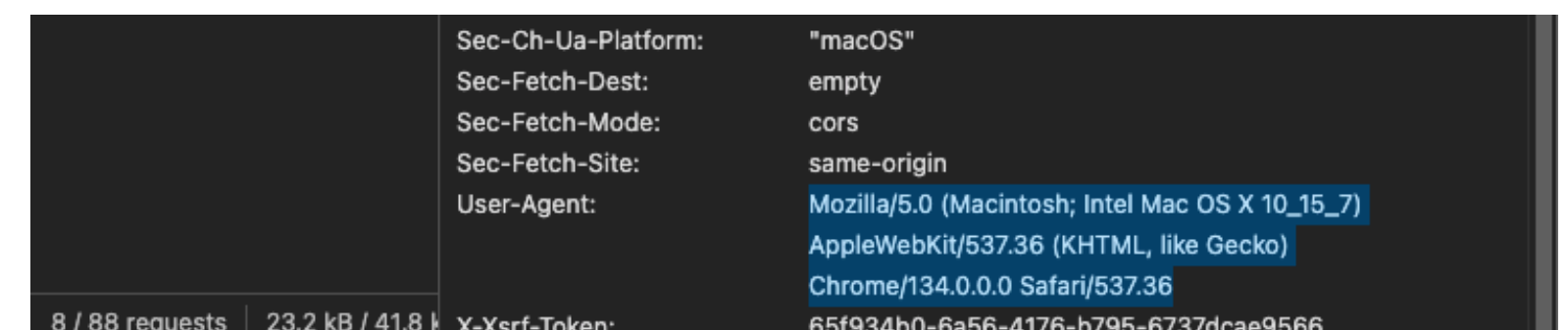
```
    file.write(response.content)
```

```
#request User-Agent 헤더 확인  
print(response.request.headers)
```

```
{'User-Agent': 'python-requests/2.31.'}
```



~



urllib.request.urlretrieve 를 사용한 다운로드

```
import requests
import os

# 파일 URL
url = "https://www.naver.com/favicon.ico"

# GET 요청 보내기
response = requests.get(url)

# 요청이 성공했는지 확인
if response.status_code == 200:
    # 파일로 저장
    if not os.path.exists("output"):
        os.makedirs("output")
    with open("output/naver.ico", "wb") as file:
        file.write(response.content)
    print("파일 다운로드 완료")
else:
    print(f"다운로드 실패: {response.status_code}")
```



```
from urllib.request import urlretrieve

# 파일 URL
url = "https://www.naver.com/favicon.ico"

# GET 요청 보내기
path, response = urlretrieve(url, "naver.ico")

# 파일 저장 경로
print("파일 저장 경로 : ", path)
```

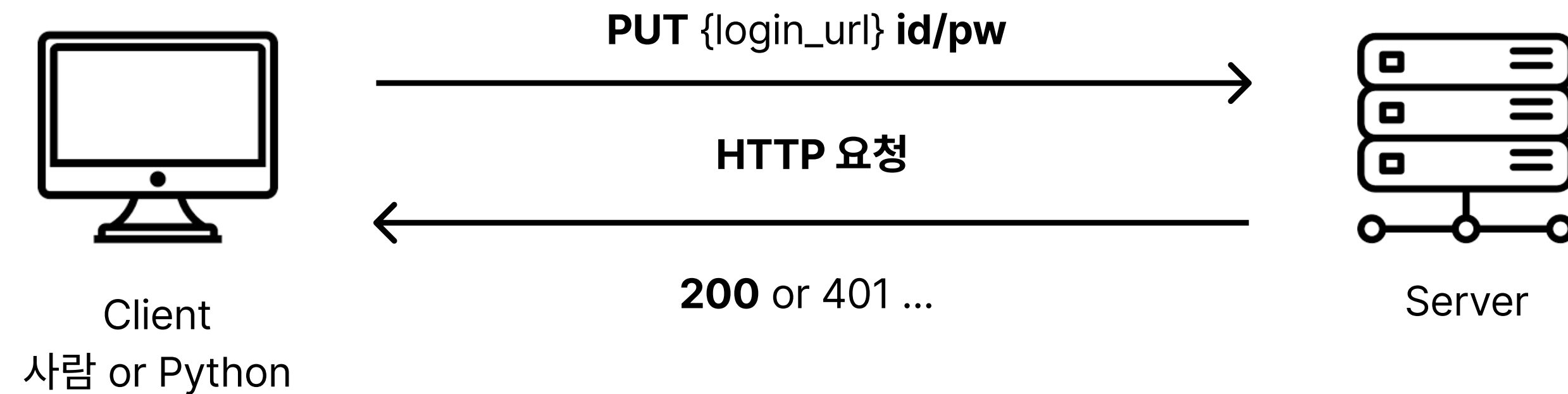
urlretrieve 단점 header 설정이 불가능하여 네이버 웹툰과 같이 User-Agent 를 검사하는 경우 사용 불가능!

로그인이 필요한 사이트 스크래핑

Session 에 대해 알아보자! 📦

로그인!

로그인은 서버에 신원 확인하는 과정을 말합니다.

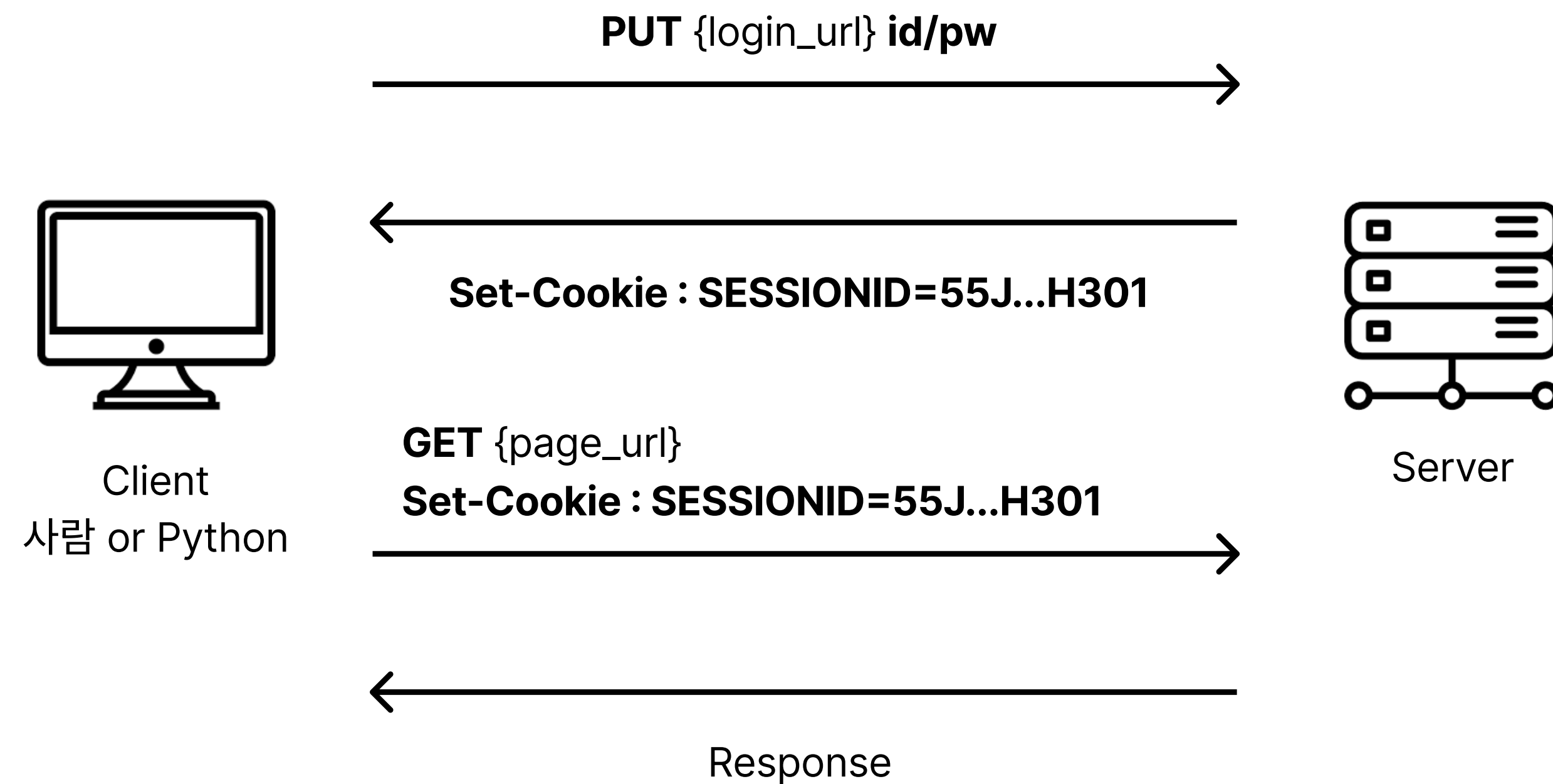


그러나 이렇게 성공 여부만 반환한다면, 일회성으로 “신원 확인”만 하게 됩니다.
따라서 로그인의 결과로 **Session**이라는 일종의 기간제 출입증을 받게 됩니다.

세션!

세션은 출입증 역할을 합니다.

이는 **응답 쿠키**에 담겨서 클라이언트에게 전달되고 클라이언트는 잘 보관하고 있다가 다음 **요청 쿠키**에 넘겨주어야 합니다.



쿠키 Cookie

각종 부가적인 (텍스트) 정보를 로컬 저장소에 저장하기 위한 텍스트 파일

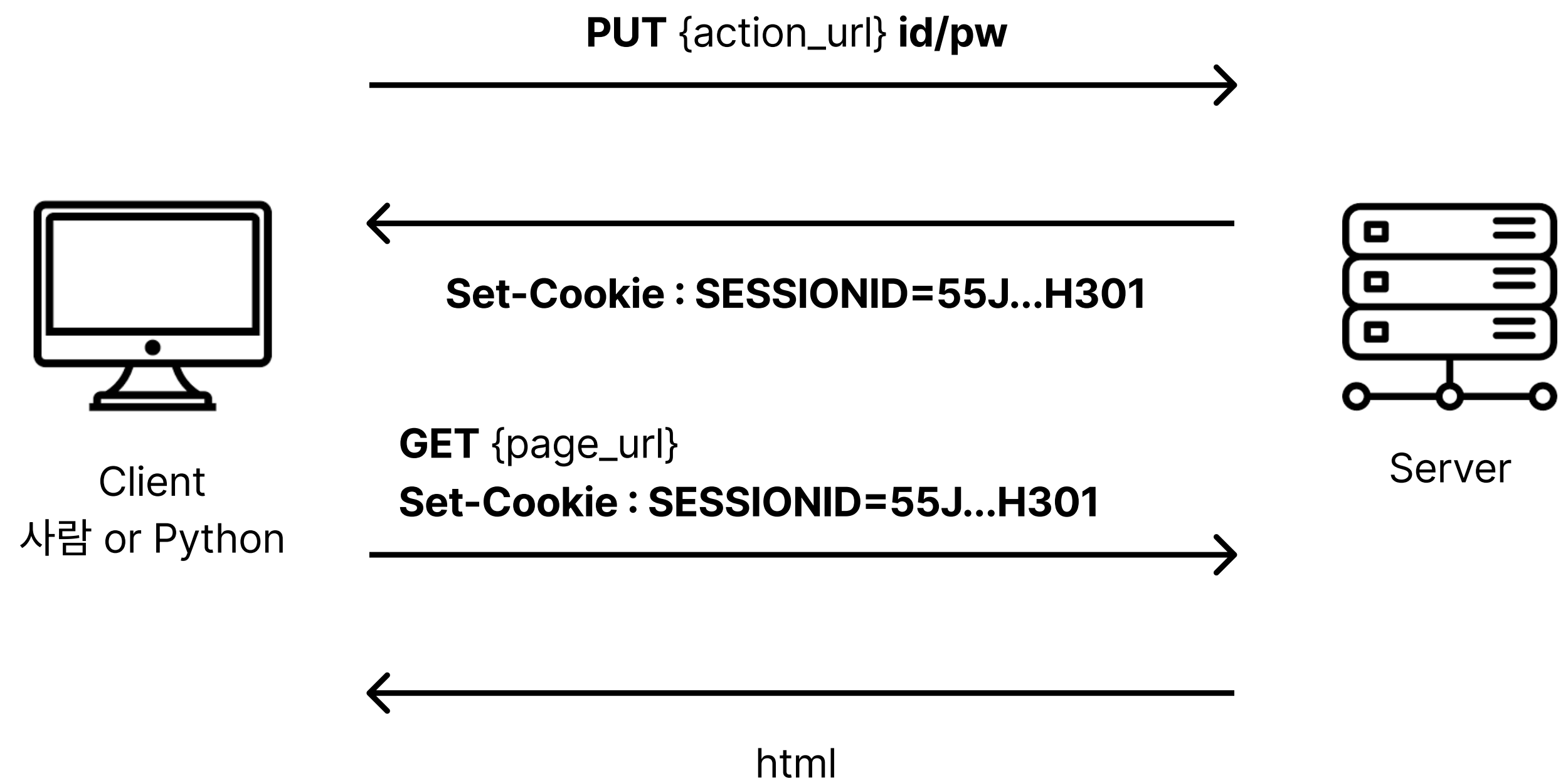
requests.Session 으로 세션 관리하기

기존의 rq 대신에 s 를 이용하면 requests 모듈이 알아서 세션을 관리해줍니다.

```
# 로그인 정보
login_info = {
    'memberID' : 'ds2023',
    'memberPassword' : '2023ds'
}

# 크롤링할 url
target_url = f"http://dowellcomputer.com/member/memberUp"

# 세션 만들기
with rq.Session() as s:
    ## action url 만들기
    res = s.get(login_url)
    soup = BeautifulSoup(res.text, 'lxml')
    action = soup.select_one('form').get('action')
    action_url = urljoin(login_url, action)
    ## login 요청
    login_res = s.post(action_url, data=login_info)
    if login_res.status_code != 200:
        raise Exception("로그인 실패")
```



Login

일반적으로 로그인 페이지에 form 태그 action 속성에 있는 값이 login url 을 나타내고 action url 이라고도 합니다.

```
<form action="./memberLoginAction.jsp" method="post"> == $0
  <div id="viewBox">Ⓜ</div>
  <div id="blackBox">Ⓜ</div>
</form>
</div>
```

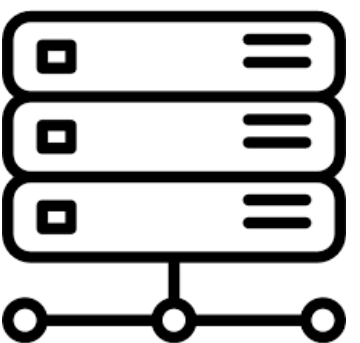
```
#세션 만들기
with rq.Session() as s:
    ## action url 만들기
    res = s.get(login_url)
    soup = BeautifulSoup(res.text, 'lxml')
    action = soup.select_one('form').get('action')
    action_url = urljoin(login_url, action)
    ## login 요청
    login_res = s.post(action_url, data=login_info)
```

```
→ print(dict(s.cookies))
```

*주의 with ... as ... : 문법이므로 해당 블록을 벗어나면 s 가 메모리에서 제거됨 (사용불가)



Client
사람 or Python



Server

GET {login_page_url}



html

PUT http://~/memberLoginAction.jsp id/pw



Set-Cookie : JSESSIONID=55J...H301

GET {page_url}

Set-Cookie : JSESSIONID=55J...H301



html

Form 태그

```
<form action="/memberLoginAction.jsp" method="post"> == $0
  <div id="viewBox">...</div>
  <div id="blackBox">...</div>
</form>
</div>
```

<input type="..." value="..." name="..."> 란?

속성

- type : 입력 태그의 유형
- value : 서버로 보낼 값
- name : 서버로 전달되는 변수명

```
<form action="/login" method="POST">
  <input type="text" value="" name="userId" >
  <input type="text" value="" name="userPassword" >
  <input type="submit" >
</form>
```

POST /login

payload

```
{
  'userId' : 'qwer1234',
  'userPassword' : 'qwer1234'
}
```

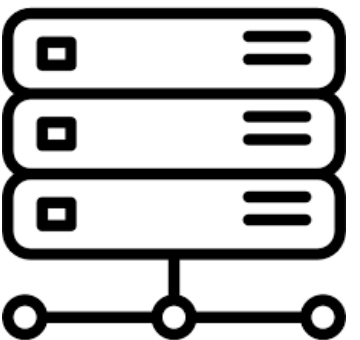
세션을 이용해서 로그인 필요한 사이트 접근

성공적으로 로그인했다면, s 객체에는 JSESSIONID 정보가 담겨 있다.

```
res = s.get(target_url)
soup = BeautifulSoup(res.text, 'lxml')
email_input = soup.select_one('input[name=memberEmail]')
email = email_input.get('value')
print(f'Email: {email}')
```



Client
사람 or Python



Server

GET {login_page_url}



html

PUT http://~/memberLoginAction.jsp id/pw



Set-Cookie : JSESSIONID=55J...H301

GET {page_url}

Set-Cookie : JSESSIONID=55J...H301



html

*주의 with ... as ... : 문법이므로 해당 블록을 벗어나면 s 가 메모리에서 제거됨 (사용불가)

다음주 ...

1. **Selenium** 으로 **Interactive** 한 사이트 스크래핑하기
2. 학교 포털 사이트 스크래핑 해보기